

Digital Systems Design Lab

(BECE102P)

Dr. Dilip Kumar Choudhary
Assistant Professor Sr. Grade 1
School of Electronics Engineering,
Vellore Institute of Technology, Vellore

ModelSim Software Download Link

<https://tinyurl.com/mvucv8yh>

The rubrics for lab as follows:

- ✓ Assessment 1 – 10 marks
- ✓ Assessment 2– 10 marks
- ✓ Assessment 3– 10 marks
- ✓ Assessment 4– 10 marks
- ✓ Assessment 5 – 10 marks
- ✓ Assessment 6 – 10 marks
- ✓ **FAT – 50 marks**

Your notebook must be clearly labelled on the cover with the following information:

Top of Form

Course Name : Digital Systems Design Laboratory

Name :

Register no :

Slot/Batch timing :

Lab faculty Name :

Theory Faculty Name :

Lab Record must contain

- Aim
- Tool Required
- Theory
- Logic Diagram
- Procedure
- Verilog Model
- Output { Screen Shot} (Result)
- Inference

Assessment-1

1. Write a Verilog model and Verify truth table for basic logic gates using
 - i. Gate level Modeling
 - ii. Data flow Modeling
2. Write a Verilog model of given expression $F(A, B, C) = \bar{A} + B\bar{C} + \bar{A}\bar{B}C$, and Verify its truth table using
 - i. Gate level Modeling
 - ii. Data flow Modeling
3. Write Verilog code for verifying half adder and half Subtractor using test bench in Modelsim..

Answer the following questions

1. Define HDL language
2. What do you mean by Verilog Module, Module declaration and Architecture Body
3. What do you mean by data type used in Verilog
4. What is the difference between Gate Level Modeling and Dataflow Modeling?

Logic Gates

- **Logic gates** are electronic circuits that operate on one or more input signals to produce an output signal
- *Gate inputs* are driven by voltages having two nominal values, e.g. 0 V and 5 V representing logic 0 and logic 1
- The *output* of a gate provides two nominal values of voltage 0 V and 5 V representing logic 0 and logic 1
- The **truth table** of a logic system (e.g. digital electronic circuit) describes the output(s) of the system for given input(s).
- In other words, a truth table is a table of all possible combinations of the variables, showing the relation between the values that the variables may take and the result of the operation.

Types of Logic gates

1) NOT/INVERTOR

2) AND

3) OR

4) BUFFER

5) XOR, XNOR

6) NAND

7) NOR

Basic logic gates

Universal logic gates (gates from which any digital circuit can be built)

Verilog HDL code

```
module logic_gates(Y_not,Y_and,Y_or,Y_buf,Y_nand,Y_nor,Y_xor,Y_xnor,A,B);
  input A,B;
  output Y_not,Y_and,Y_or,Y_buf,Y_nand,Y_nor,Y_xor,Y_xnor;

  // not gate
  not g0(Y_not,A);

  //and gate
  and g1(Y_and,A,B);

  //or gate
  or g2(Y_or,A,B);

  //buffer gate
  buf g3(Y_buf,A);

  //nand gate
  nand g4(Y_nand,A,B);

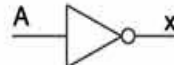
  //nor gate
  nor g5(Y_nor,A,B);

  //xor gate
  xor g6(Y_xor,A,B);

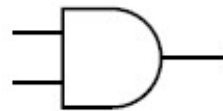
  //xnor gate
  xnor g7(Y_xnor,A,B);

endmodule
```

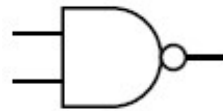
Logic Gates

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR																																																																																																
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A + B$	$\overline{A + B}$	$A \oplus B$	$\overline{A \oplus B}$																																																																																																
Symbol																																																																																																							
Truth Table	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	0	1	0	0	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	1
A	X																																																																																																						
0	1																																																																																																						
1	0																																																																																																						
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					

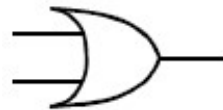
Verilog Primitive gates



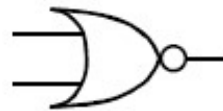
and(out, in1, in2)



nand(out, in1, in2)



or(out, in1, in2)



nor(out, in1, in2)



xor(out, in1, in2)



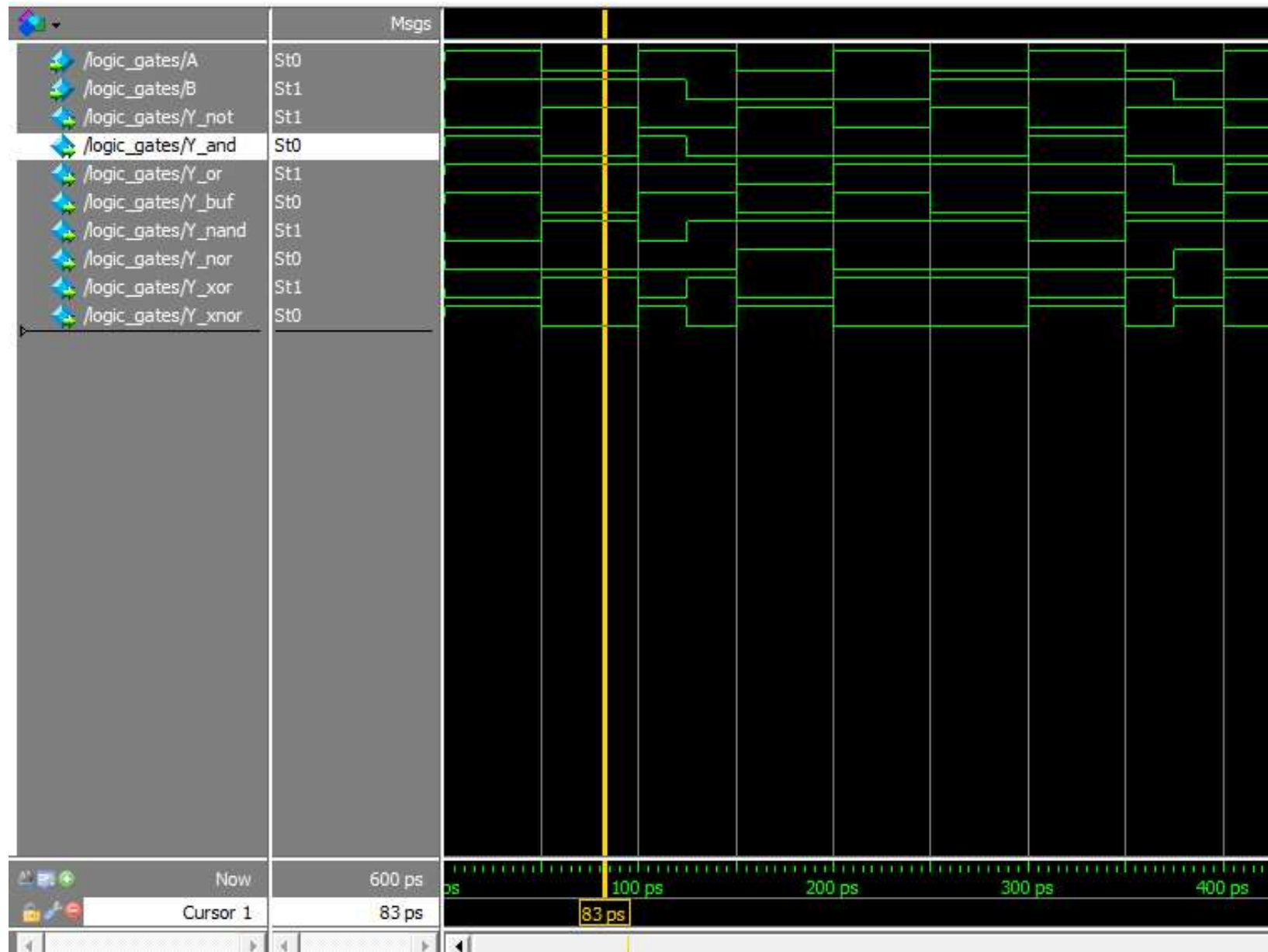
xnor(out, in1, in2)



buf(out, in)



not(out, in)




```

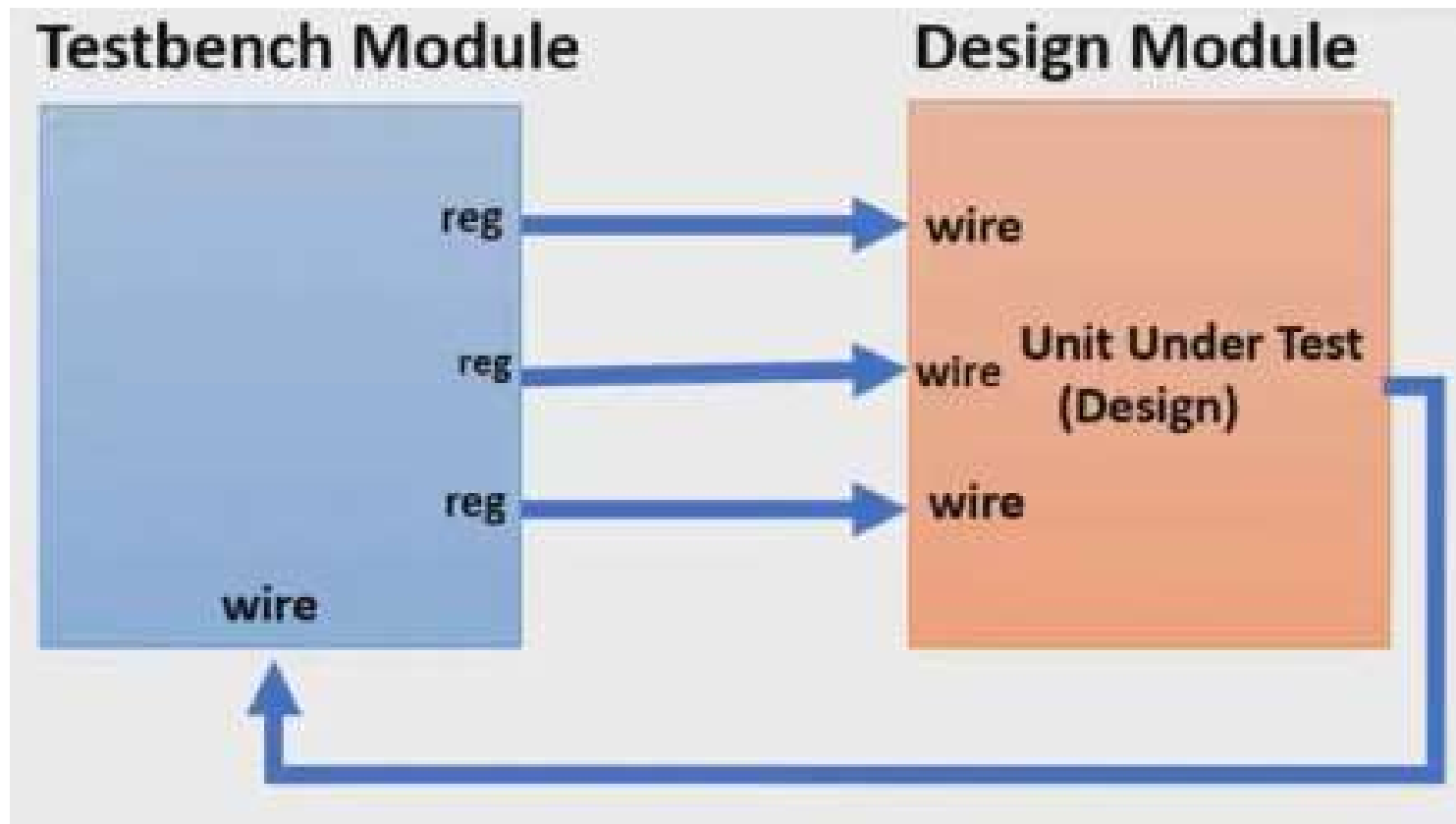
module ex C f, A1, B, C);
    input  A, B, C;
    output f;
    wire  w1, w2, w3, w4, w5;

    not  m0 (w1, A);
    not  m1 (w2, B);
    and  m2 (w3, C);

    and  m3 (w4, B, w1);
    and  m4 (w5, w1, C, w4);
    or   m5 (f, w5, w4, w4);
endmodule

```

Testbench in Verilog HDL



- Each signal in Verilog belongs to either a **net** or a **register**
- A **net** (**wire**) represents a physical wire. Its signal value is determined by its driver.
- If it is not driven by any driver, its value is high impedance (Z).
- A **register** is like a variable in programming languages. It keeps its value until a new value is assigned to it.
- Unlike registers, nets do not have storage capacity.

Description of the Circuit:

Half Adder and half subtractor are the examples of combinational circuits

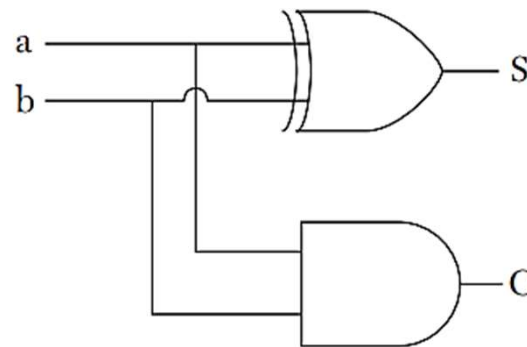
Half adder:

Half adder is used to add two single bits. The half adder takes two input of 1 bit each. It generates two outputs – Sum and Carry. Sum is represented by S and Carry is represented by C_{out} .

Truth table:

Inputs		Outputs	
A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

K – mapping:



For Carry

A \ B	0	1
0	0	0
1	0	1

$$\text{Carry} = AB$$

For Sum

A \ B	0	1
0	0	1
1	1	0

$$\begin{aligned}\text{Sum} &= A\bar{B} + \bar{A}B \\ &= A \oplus B\end{aligned}$$

Code:

```

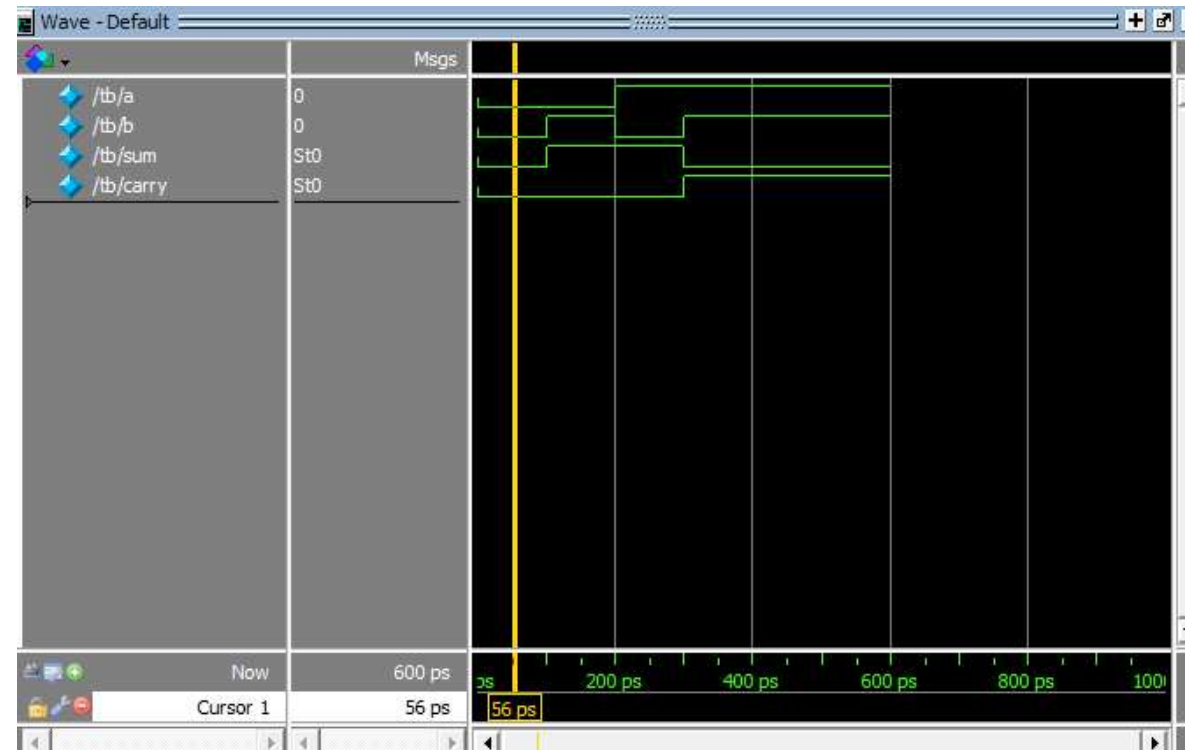
module HalfAdder(a,b,sum,carry);
input a,b;
output sum,carry;
xor(sum, a,b);
and (carry,a,b);
endmodule

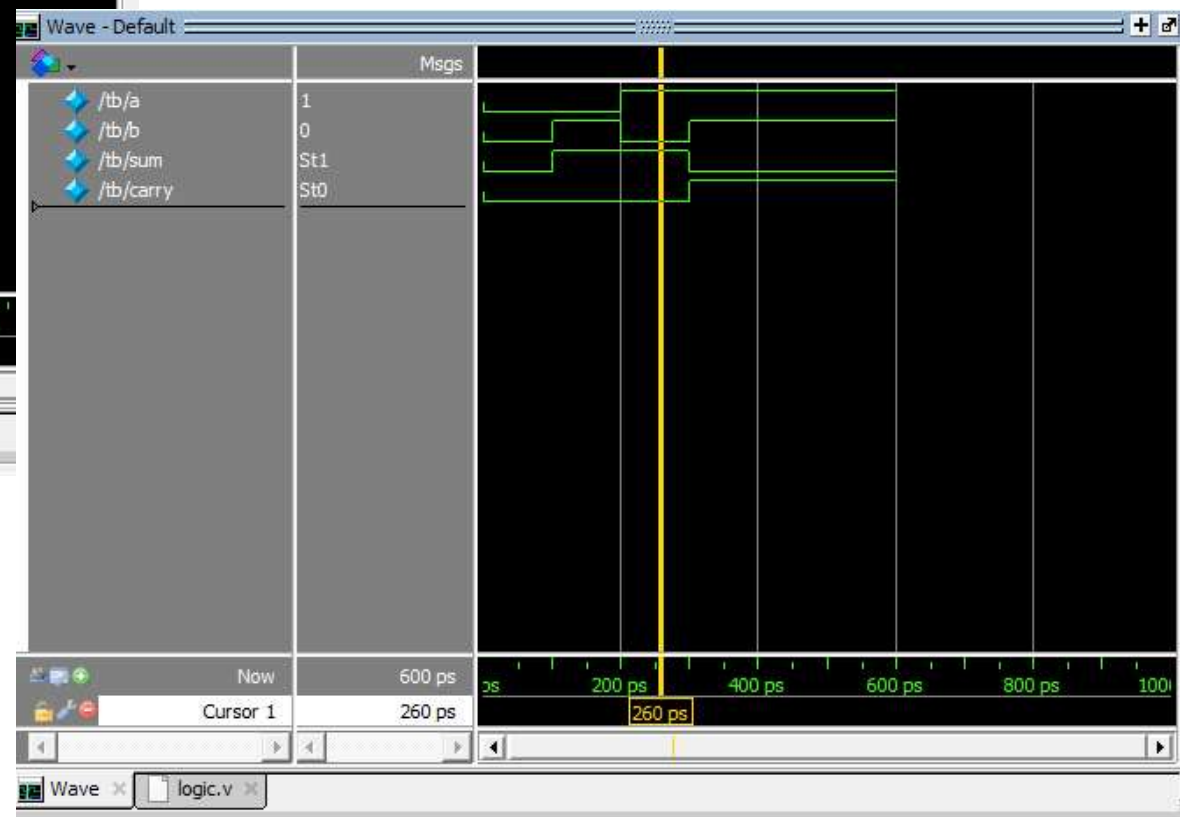
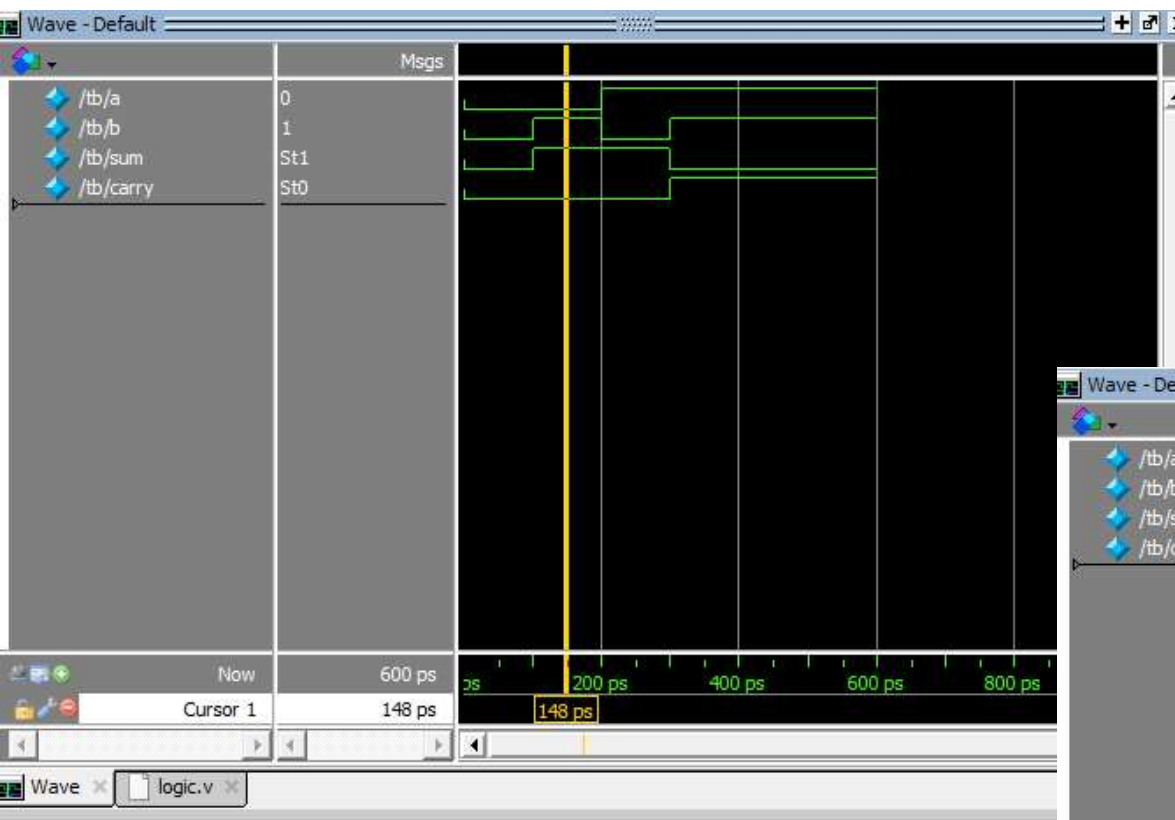
```

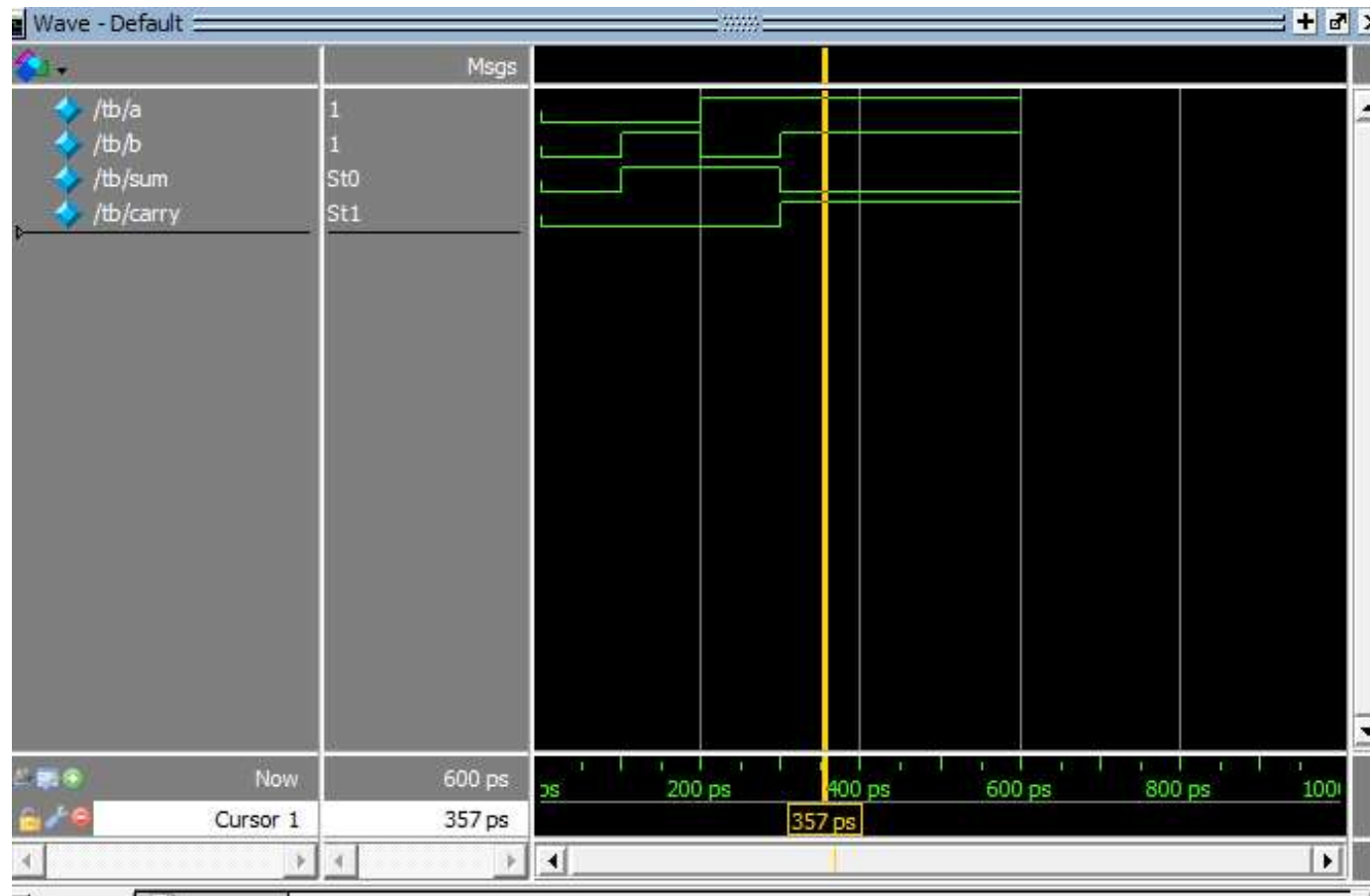
```

module tb();
reg a,b;
wire sum,carry;
HalfAdder mo(a,b,sum,carry);
initial
begin
a = 1'b0; b = 1'b0;
#100 a = 1'b0; b = 1'b1;
#100 a = 1'b1; b = 1'b0;
#100 a = 1'b1; b = 1'b1;
end
endmodule

```





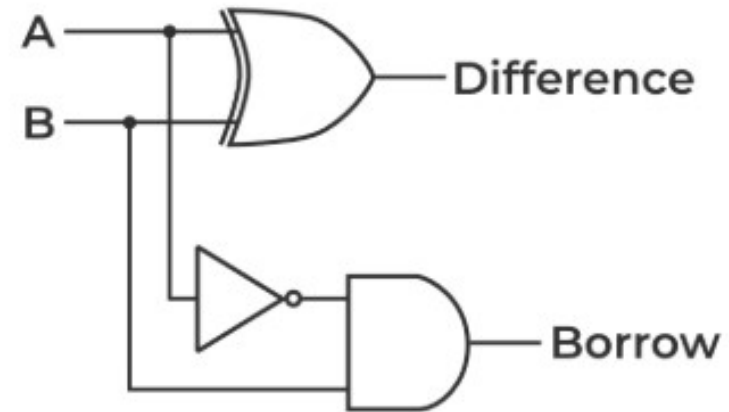


Half subtractor:

A half subtractor is a digital logic circuit that performs binary subtraction of two single-bit binary numbers. It has two inputs, A and B, and two outputs, DIFFERENCE and BORROW.

Truth table:

Inputs		Outputs	
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



K - mapping:

A \ B	$\bar{B}$	B
$\bar{A}$	0	1
A	1	0

$$\Sigma = A \oplus B$$

A \ B	$\bar{B}$	B
$\bar{A}$	0	1
A	0	0

$$B_{out} = \bar{A} \cdot B$$

```

Code: module Halfsubtractor(a,b,difference,borrow);
input a,b;
output difference,borrow;
xor(difference, a,b);
assign borrow = (~a&b);
endmodule

module tb_s();
reg a,b;
wire difference,borrow;
Halfsubtractor mo(a,b,difference,borrow);
initial
begin
a = 1'b0; b = 1'b0;
#100 a = 1'b0; b = 1'b1;
#100 a = 1'b1; b = 1'b0;
#100 a = 1'b1; b = 1'b1;
end
endmodule

```

